

Linux 네트워크 개념 정리

0. Overview

Linux 네트워크 스택은 매우 다양하고 복잡한 것 같습니다.. 그래서 Network Namespace, veth, Bridge, Netfilter 등의 개념들이 정확히 어떤 역할을 하는지 이해하기가 조금 힘들었습니다.

그래서 이번에는 OSI 7계층에 대해서 각 기술은 어떤 계층에 속하는지를 추가해서 정리해보았습니다.

1. OSI 7계층 개요

OSI(Open Systems Interconnection) 모델은 네트워크 통신을 7개의 계층으로 나눈 참조 모델입니다.

계층	이름	주요 역할	데이터 단위(PDU)	대표 프로토콜/기술
L7	응용 계층 (Application)	사용자 애플리케이션에 서비스 제공	메시지	HTTP, DNS, SSH
L6	표현 계층 (Presentation)	데이터 형식 변환, 암호화, 압축	메시지	TLS/SSL, 인코딩
L5	세션 계층 (Session)	연결 세션 수립/유지/종료	메시지	소켓 세션, RPC
L4	전송 계층 (Transport)	종단 간 신뢰성, 포트 기반 다중화	세그먼트(TCP)/데이터그램(UDP)	TCP, UDP
L3	네트워크 계층 (Network)	논리 주소 기반 경로 결정 (라우팅)	패킷	IP, ICMP, 라우팅
L2	데이터링크 계층 (Data Link)	동일 네트워크 내 프레임 전달, MAC 주소	프레임	Ethernet, ARP(경계), Bridge
L1	물리 계층 (Physical)	비트를 실제 신호(전기, 광, 전파)로 전송	비트	케이블, NIC 하드웨어

이제 이 계층 순서를 따라, 각 계층에 속하는 Linux Network 기술을 살펴보도록 하겠습니다. 단, Network Namespace는 특정 계층에 속하는 기술이 아니라 L2~L4 전체 스택을 통째로 복제&격리하는 개념이므로, 계층별 설명에 앞서 먼저 다뤄보겠습니다.

2. Network Namespace

Network Namespace(netns)는 Linux 커널의 namespace 격리 기능 중 하나로, 특정 계층에 국한되지 않고 네트워크 스택 전체(L2 인터페이스, L3 라우팅 테이블, L3/L4에 걸친 방화벽 규칙, ARP 테이블, 소켓 등)를 프로세스 그룹 단위로 통째로 복제하여 격리합니다. 컨테이너 기술(Docker, Kubernetes Pod 등)이 각 컨테이너가 독립된 네트워크를 가진 것처럼 보이게 만드는 핵심 메커니즘이 바로 이 Network Namespace라고 할 수 있습니다.

공식 문서는 이를 다음과 같이 정의합니다.

A network namespace is logically another copy of the network stack, with its own routes, firewall rules, and network devices. - [ip-netns\(8\) man page](#)

기본적으로 프로세스는 부모로부터 네트워크 네임스페이스를 상속받으며, 초기에는 모든 프로세스가 init 프로세스의 기본 네트워크 네임스페이스를 공유합니다. (Root Namespace 라고도 부른다고 합니다)

2-1. 주요 명령어 (iproute2 `ip netns` 기준)

```
# 네임스페이스 생성
sudo ip netns add ns1
sudo ip netns add ns2

# 네임스페이스 목록 조회
ip netns list

# 네임스페이스 내부에서 명령 실행
sudo ip netns exec ns1 ip addr show

# 네임스페이스 내부 쉘 진입
sudo ip netns exec ns1 bash

# 네임스페이스 삭제
sudo ip netns delete ns1
```

이름이 지정된 네임스페이스는 관례적으로 `/var/run/netns/NAME` 에 바인드 마운트된 파일로 존재합니다. Linux에서는 "모든 것은 파일이다" 라는 철칙에 따라 네임스페이스도 파일 형식으로 다룰 수 있도록 해두었습니다. 이 파일을 열어서 얻은 파일 디스크립터는 `setns(2)` 시스템 콜로 프로세스의 네트워크 네임스페이스를 전환하는 데 사용할 수 있습니다.

2-2. 격리되는 자원

계층	격리되는 자원	격리 안 되는 것(다른 namespace 필요)
L2	인터페이스(veth, bridge 등), MAC/ARP 테이블	—
L3	라우팅 테이블(FIB), IP 주소	—
L3/L4	iptables/nftables 규칙, conntrack 테이블	—
L4~L7	포트 네임스페이스, 소켓	파일시스템(mount ns), PID(pid ns), 사용자(user ns)

컨테이너 런타임은 보통 network ns를 mount ns, pid ns, uts ns, user ns 등과 함께 조합하여 사용합니다.

3. L1 물리 계층 (Physical Layer)

물리 계층은 비트를 실제 전기 신호, 광 신호, 전파로 변환해 전송하는 계층입니다. Linux 커널 관점에서 NIC(Network Interface Card) 드라이버가 이 계층을 담당하며, 커널 소프트웨어 스택에서 직접 다룰 일은 많지 않지만 아래 개념들은 알고 가면 좋을 것 같습니다.

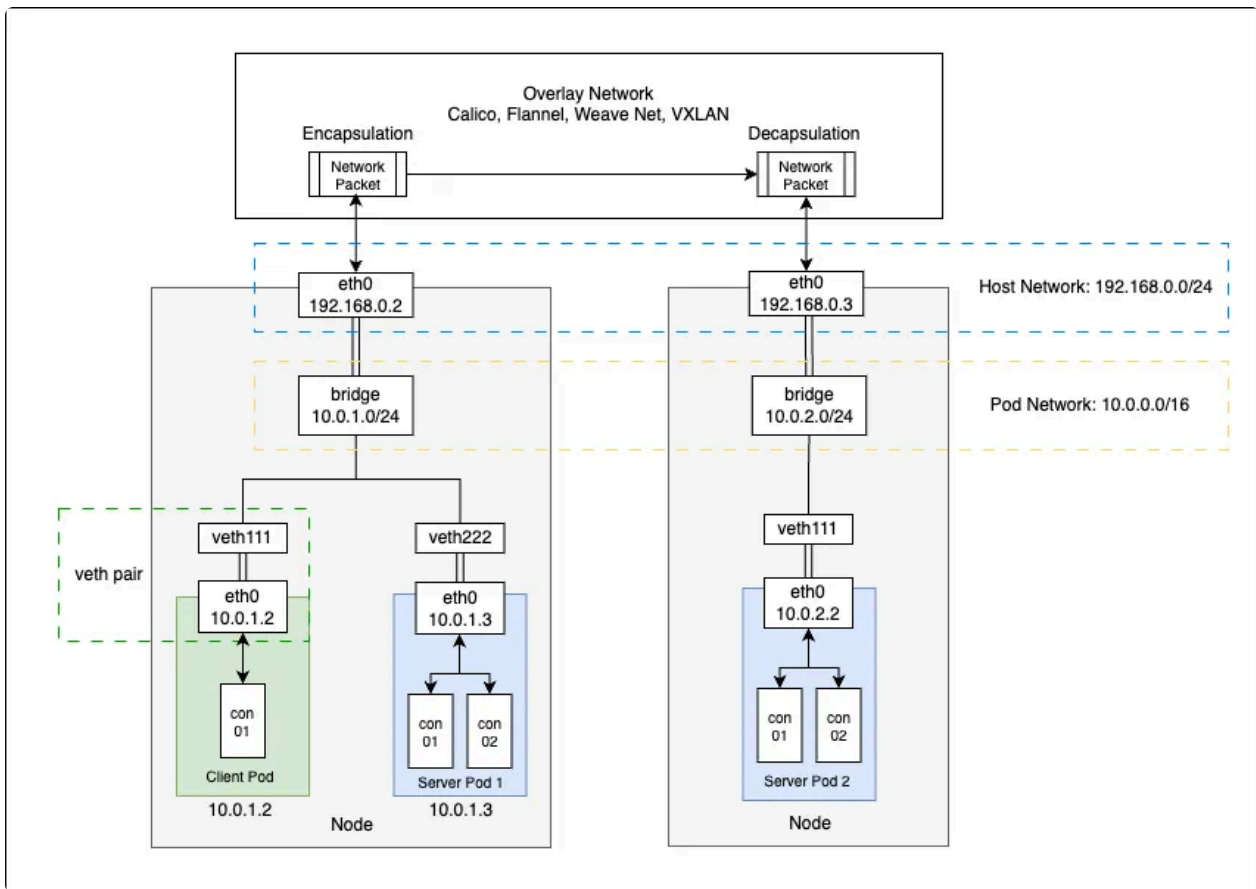
- **net_device** 구조체: 커널이 물리/가상 NIC를 표현하는 자료구조로, 이후 다룰 veth, bridge 같은 가상 장치도 결국 이 구조체로 표현됩니다.
- **sk_buff(socket buffer)**: 패킷이 스택을 오르내리는 동안 사용되는 커널 내부 표현 구조체로, 헤더 포인터만 조정하며 데이터 복사를 최소화합니다. L1에서 수신된 비트열이 프레임으로 조립되어 sk_buff에 담기는 순간부터 커널 네트워크 스택의 처리가 시작됩니다.

4. L2 데이터링크 계층 (Data Link Layer)

L2는 같은 네트워크(브로드캐스트 도메인) 안에서 **MAC** 주소를 기준으로 프레임을 주고받는 계층입니다. 이 문서에서 다루는 veth와 Linux Bridge가 바로 이 계층의 핵심 기술입니다.

4.1 veth (Virtual Ethernet Pair)

veth는 항상 쌍(pair)으로 생성되는 가상 이더넷 장치입니다. 한쪽으로 들어간 프레임은 즉시 반대쪽으로 나오는, 마치 양쪽 끝에 연결된 가상 케이블처럼 동작합니다.



공식 문서 설명:

The veth devices are virtual Ethernet devices. They can act as tunnels between network namespaces to create a bridge to a physical network device in another namespace, but can also be used as standalone network devices. veth devices are always created in interconnected pairs. - [veth\(4\) man page](#)

Packets transmitted on one device in the pair are immediately received on the other device. When either devices is down the link state of the pair is down. - [veth\(4\) man page](#)

veth 생성 및 네임스페이스 이동은 아래 명령어로 할 수 있습니다.

```
# veth pair 생성 (호스트 기본 네임스페이스에 둘 다 생성됨)
sudo ip link add veth-host type veth peer name veth-ns1

# 한쪽을 ns1 네임스페이스로 이동
sudo ip link set veth-ns1 netns ns1

# 호스트 측 설정 및 활성화
sudo ip addr add 10.0.0.1/24 dev veth-host
sudo ip link set veth-host up

# 네임스페이스 내부 측 설정
```

```
sudo ip netns exec ns1 ip addr add 10.0.0.2/24 dev veth-ns1
sudo ip netns exec ns1 ip link set veth-ns1 up
sudo ip netns exec ns1 ip link set lo up
```

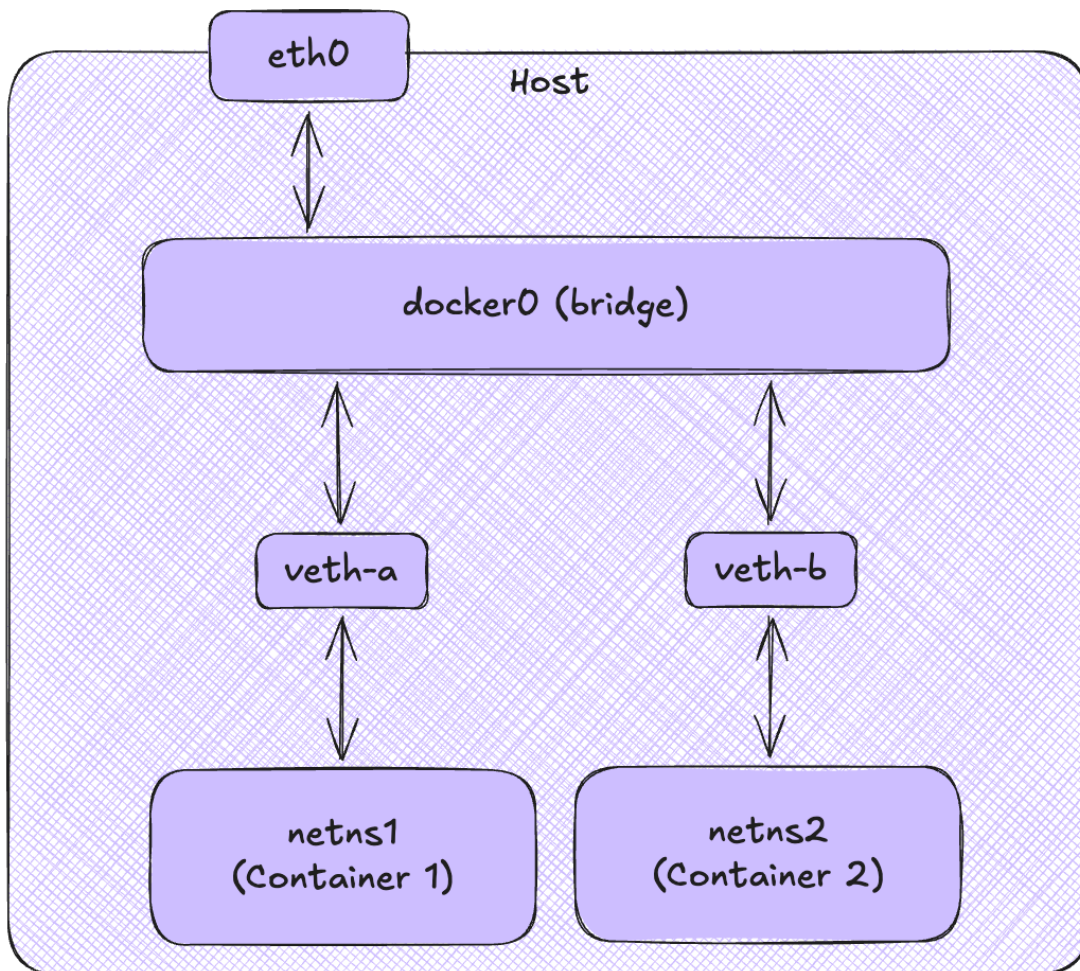
veth 쌍 하나를 한쪽 네트워크 네임스페이스에, 다른 하나를 또 다른 네임스페이스에 배치하는 것은 네임스페이스 간 L2 통신을 가능하게 만듭니다. 이것이 컨테이너를 호스트 네트워크에 연결하는 기본 원리라고 할 수 있습니다. Docker의 기본 브리지 네트워크, Kubernetes CNI 플러그인(bridge, flannel 등) 대부분이 veth pair 한쪽은 컨테이너 netns 안에 연결하고, 다른 한쪽은 호스트의 브리지에 연결하는 패턴을 많이 사용합니다.

4.2 Linux Bridge

Linux Bridge는 소프트웨어로 구현된 **L2** 스위치입니다. 여러 개의 네트워크 인터페이스(물리 NIC, veth, tap 등)를 하나의 브로드캐스트 도메인으로 묶어주며, MAC 주소 학습(learning), 플러딩, STP(Spanning Tree Protocol) 등 실제 이더넷 스위치와 동일한 동작을 커널 안에서 수행합니다.

브리지에 연결된 각 인터페이스는 포트(port)라고 부릅니다. 포트는 물리 NIC일 수도 있고 veth나 tap 같은 가상 인터페이스일 수도 있으며, 브리지 입장에서는 종류와 무관하게 프레임을 주고받는 하나의 창구로 취급됩니다. `ip link set <iface> master <bridge>` 명령으로 인터페이스를 브리지에 편입시키면, 그 인터페이스는 더 이상 독립적인 L3 네트워크 장치로 동작하지 않고 브리지의 포트만 기능합니다(IP 주소는 개별 인터페이스가 아니라 브리지 장치 자체에 부여하는 것이 일반적입니다). 브리지는 이렇게 등록된 포트마다 별도의 상태(포워딩 여부, STP 상태 등)를 관리하며, FDB의 각 엔트리도 결국 "어떤 MAC이 어떤 포트 뒤에 있는가"를 나타내는 대응 관계입니다.

컨테이너 네트워킹에서 일반적인 구성은 다음과 같습니다.



Bridge는 아래와 같이 동작합니다.

1. **Learning**: 브리지는 각 포트에 들어오는 프레임의 출발지 MAC 주소를 보고 "이 MAC은 이 포트 뒤에 있다"는 것을 FDB(Forwarding Database)에 기록합니다.
2. **Flooding**: 목적지 MAC이 FDB에 없으면 모든 포트에 뿌립니다(브로드캐스트 포함).
3. **Forwarding**: 목적지 MAC이 FDB에 있으면 해당 포트에 전달합니다.
4. **STP**: 루프 방지를 위한 스페닝 트리 프로토콜을 지원하며, `bridge-utils`의 `brctl` 또는 최신 `ip link/bridge` 명령으로 설정합니다.

위 동작 원리를 조금 더 자세히 보겠습니다.

Bridge가 처음 생성되면 어떤 MAC 주소가 어떤 포트에 연결되어 있는지 모릅니다. 하지만 프레임이 도착하면, 프레임 안에는 발신 호스트의 MAC 주소가 적혀 있습니다. 이것을 보고 Bridge는 FDB를 업데이트합니다.

만약 Bridge가 특정 MAC 주소에 프레임을 보내려고 하는데 목적지의 MAC 주소가 FDB에 없을 수 있습니다. 이 경우에는 모든 포트에 연결되어 있는 인터페이스에 뿌립니다.

만약 프레임을 보내야하는 목적지 MAC 주소가 FDB에 있다면, 해당 포트로만 전달할 수 있습니다.

STP는 Spanning Tree Protocol의 약자로, 여러 Bridge를 연결하는 경우 Cycle이 발생할 수 있습니다. Cycle이 발생하게 되면 프레임이 무한 루프에 바질 위험이 있으므로 이를 미리 막아두는 규칙입니다.

5. L2 - L3 사이? ARP

ARP(Address Resolution Protocol)는 L3의 논리 주소(IP)를 L2의 물리 주소(MAC)로 변환하는 프로토콜입니다. 즉 ARP 자체는 L3 주소를 다루지만 결과물은 L2 프레임을 만드는 데 쓰이기 때문에, 흔히 L2와 L3의 경계(또는 2.5계층)에 있다고 설명하곤 합니다.

Linux 커널의 ARP 모듈은 다음과 같이 설명됩니다.

This kernel protocol module implements the Address Resolution Protocol defined in RFC 826. It is used to convert between Layer2 hardware addresses and IPv4 protocol addresses on directly connected networks. The user normally doesn't interact directly with this module except to configure it; instead it provides a service for other protocols in the kernel. - [arp\(7\) — Linux manual page](#)

동작 방식(요청-응답 기반):

Essentially, the ARP protocol is request-reply based. If a host wishes to translate an IP address into an Ethernet address, it will send an ARP request to all hosts on the local network, using an Ethernet broadcast. - [Networking basics – IP routing and the ARP protocol](#)

5.1 ARP 캐시(ARP Cache)

ARP 캐시는 이미 알아낸 IP 주소 ↔ MAC 주소 매핑을 임시로 저장해두는 테이블입니다. 통신할 때마다 매번 ARP 요청을 브로드캐스트로 보내면 네트워크에 불필요한 트래픽이 계속 발생하므로, 한번 알아낸 매핑은 캐시에 담아두고 재사용합니다.

캐시의 각 엔트리는 대략 다음 정보를 담고 있습니다.

- IP 주소
- 그에 대응하는 MAC 주소

- 어느 인터페이스에서 학습되었는지
- 현재 상태(state)

```
$ ip neigh show
192.168.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
192.168.1.50 dev eth0 lladdr 11:22:33:44:55:66 STALE
```

상태(state) 전이

ARP 캐시 엔트리는 고정된 값이 아니라 시간에 따라 여러 상태를 거치며 관리됩니다. 주요 상태는 다음과 같습니다.

상태	의미
INCOMPLETE	ARP 요청을 보냈지만 아직 응답을 받지 못한 상태
REACHABLE	최근에 응답을 확인해 유효하다고 판단되는 상태
STALE	일정 시간이 지나 유효성이 불확실해진 상태(다음 통신 시 재확인 필요)
DELAY	STALE 상태에서 실제로 트래픽이 발생해 재확인을 준비 중인 상태
PROBE	재확인을 위해 해당 호스트에게 유니캐스트로 ARP 요청을 보낸 상태
FAILED	재확인에 실패해 더 이상 유효하지 않다고 판단된 상태
PERMANENT / NOARP	관리자가 수동으로 등록해 ARP로 갱신되지 않는 정적 엔트리

즉 엔트리는 INCOMPLETE → REACHABLE → STALE → DELAY/PROBE → REACHABLE(재확인 성공) 또는 FAILED(재확인 실패)의 흐름을 반복하며 관리됩니다. 이렇게 상태를 단계적으로 나눠두는 이유는, 매번 즉시 만료시키고 새로 ARP 요청을 보내는 대신 "일단 STALE 상태로 두었다가 실제로 그 호스트와 통신이 필요해지는 시점에만" 재확인을 시도해서 불필요한 ARP 트래픽을 줄이기 위해서입니다.

5.2 Gratuitous ARP

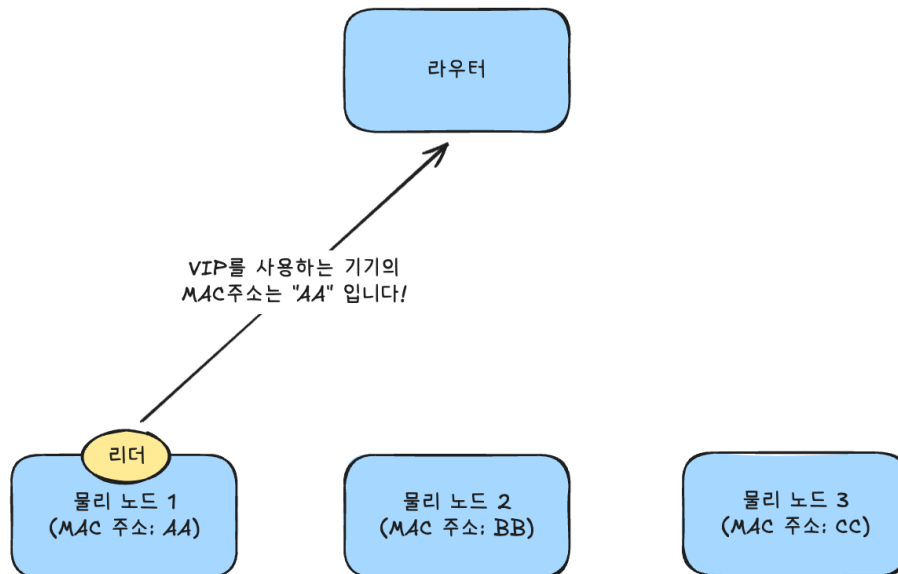
영어 [편집]

- 어원:
 1. (못마땅함) 불필요한, 쓸데없는,
 2. 무상의

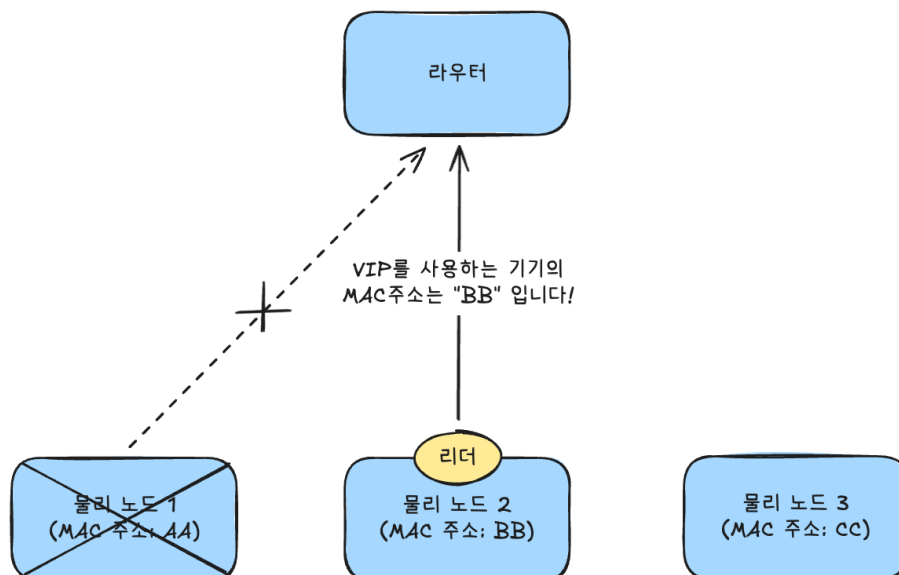
호스트가 자신의 IP-MAC 매핑을 스스로 네트워크에 알리기 위해 요청도 받지 않았는데 먼저 ARP 응답(또는 자기 자신을 목적지로 하는 ARP 요청)을 브로드캐스트하는 경우가 있는데, 이를 Gratuitous ARP라고 부릅니다. 인터페이스에 새 IP가 할당되었을 때, 혹은 페일오버(failover)로 IP가 다른 장비로 옮겨갔을 때 주변 호스트들의 ARP 캐시를 최신 상태로 갱신시키기 위한 용도로 사용됩니다.

Gratuitous ARP를 사용하는 대표적인 예시로 Kube-vip가 있습니다. Kube-vip를 사용하면 Gratuitous ARP를 통해 여러 호스트가 하나의 IP를 Failover 할 수 있게 됩니다.

1. 노드 1이 리더로 선출됨



2. 노드 1 장애 발생, 노드 2가 리더를 이어 받음



5.3 명령어

```
# ARP/이웃 캐시 확인 (IPv4 + IPv6 통합, 최신 방식)
```

```
ip neigh show
```

```
# 전통적 방식
```

```
arp -a
```

```
# 정적 ARP 엔트리 추가
```

```
sudo ip neigh add 192.168.1.50 lladdr aa:bb:cc:dd:ee:ff dev eth0

# ARP 캐시 삭제
sudo ip neigh flush dev eth0
```

6. L3 네트워크 계층 (Network Layer)

L3는 논리 주소(IP)를 기준으로 목적지까지의 경로를 결정하는 계층입니다. IP, ICMP, 그리고 라우팅이 이 계층의 핵심입니다.

6.1 라우팅 테이블(FIB, Forwarding Information Base)

커널은 목적지 IP를 보고 어느 인터페이스로 패킷을 보낼지를 라우팅 테이블(FIB)을 통해 결정합니다.

```
# 라우팅 테이블 확인
ip route show
# 예: default via 192.168.1.1 dev eth0
# 192.168.1.0/24 dev eth0 proto kernel scope link src
192.168.1.100

# 특정 목적지에 대한 실제 라우팅 결정 확인
ip route get 8.8.8.8

# 정적 경로 추가
sudo ip route add 10.0.1.0/24 via 10.0.0.254 dev eth0

# 기본 게이트웨이 설정
sudo ip route add default via 192.168.1.1
```

`ip route show` 출력의 각 줄은 하나의 경로(route)를 나타내며, 다음과 같은 필드로 구성됩니다.

필드	의미
목적지 (예: 192.168.1.0/24 , default)	이 경로가 적용되는 목적지 네트워크. default 는 0.0.0.0/0 , 즉 다른 어떤 경로에도 매치되지 않을 때 쓰이는 최종 경로
via	다음 홉(next hop)의 IP 주소. 같은 네트워크 안이 아니라 다른 라우터를 거쳐야 할 때 지정
dev	패킷을 내보낼 인터페이스
proto	이 경로가 어떻게 등록되었는지(kernel : 인터페이스 설정 시 자동 생성, static : 관리자가 직접 추가, dhcp : DHCP로 받음 등)

필드	의미
scope	경로의 유효 범위(link: 같은 링크 내 직접 연결, global: 원격 네트워크 등)
src	이 경로로 나갈 때 사용할 출발지 IP 주소
metric	같은 목적지에 여러 경로가 있을 때의 우선순위(값이 낮을수록 우선)

예를 들어 `192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100`은 "192.168.1.0/24 네트워크는 eth0에 직접 연결되어 있고(scope link), 이 경로로 나갈 때는 출발지 주소로 192.168.1.100을 쓴다"는 뜻이며, 인터페이스에 IP를 설정하는 순간 커널이 자동으로 만들어 넣은 경로입니다.

라우팅 결정은 최장 프리픽스 매칭 원칙을 따릅니다. 즉 여러 경로가 목적지에 매치될 수 있을 때 가장 구체적인(서브넷 마스크가 가장 긴) 경로가 우선합니다. 예를 들어 라우팅 테이블에 다음 세 경로가 동시에 존재한다고 가정해보면..

```
default via 192.168.1.1 dev eth0          # 0.0.0.0/0
10.0.0.0/8 via 192.168.1.254 dev eth0      # /8
10.0.1.0/24 via 192.168.1.253 dev eth0     # /24
```

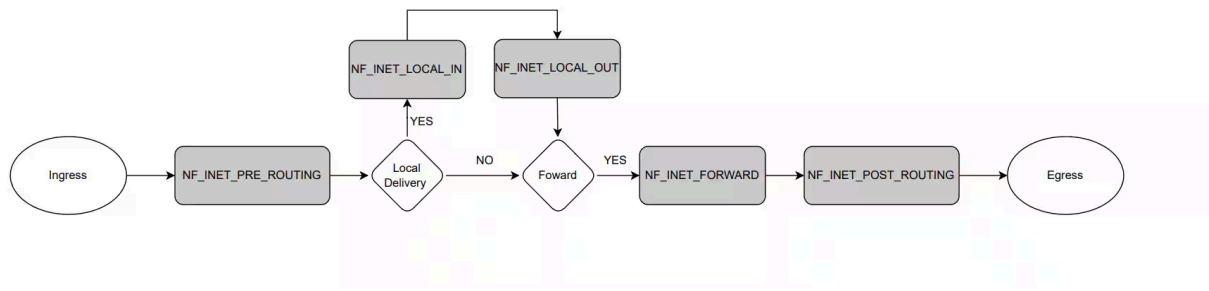
목적지가 `10.0.1.5` 인 패킷은 세 경로 모두에 매치되지만(`0.0.0.0/0`, `10.0.0.0/8`, `10.0.1.0/24` 전부 `10.0.1.5`를 포함), 커널은 그중 Prefix 길이가 가장 긴 `10.0.1.0/24` 경로를 선택해 `192.168.1.253` 을 다음 홉으로 사용합니다. `default` 경로(/0)는 다른 어떤 경로와도 매치되지 않을 때만 쓰이는 가장 우선순위가 낮은 게이트웨이입니다.

7. L3 - L4: Netfilter / iptables / nftables

Netfilter는 특정 계층 하나에 속하지 않고, L3(IP 헤더)와 L4(TCP/UDP 포트) 정보를 함께 검사하며 커널 네트워크 스택 곳곳에 정의된 **훅(hook)** 지점에 콜백 함수를 등록할 수 있게 해주는 프레임워크입니다. iptables, nftables, conntrack, NAT 엔진이 모두 이 프레임워크 위에서 동작합니다.

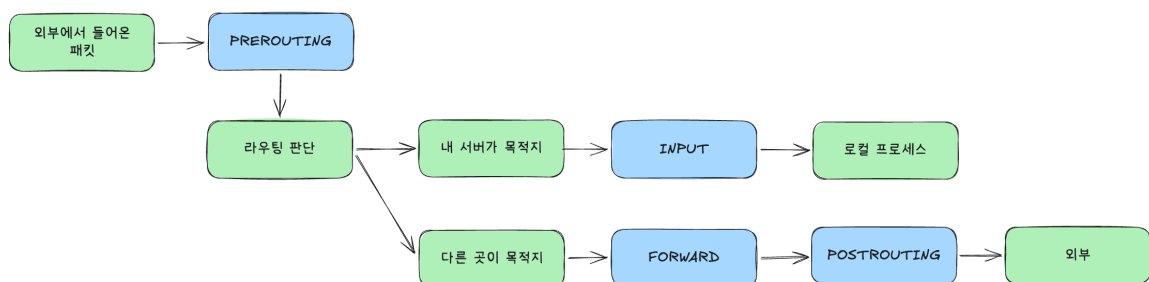
Netfilter represents a set of hooks inside the Linux kernel, allowing specific kernel modules to register callback functions with the kernel's networking stack. Those functions, usually applied to the traffic in the form of filtering and modification rules, are called for every packet that traverses the respective hook within the networking stack. - [Netfilter — Wikipedia](#)

7.1 훅(Chain) 종류



Chain	실행 시점	주된 용도
PREROUTING	패킷이 네트워크 인터페이스로 들어온 직후, 라우팅 결정 전	목적지 주소 변경(DNAT)
INPUT	이 서버 자신이 최종 목적지인 패킷	서버로 들어오는 접속 허용/차단
FORWARD	이 서버를 거쳐 다른 곳으로 전달되는 패킷	라우터·게이트웨이 트래픽 제어
OUTPUT	이 서버가 직접 생성해 외부로 보내는 패킷	서버발 통신 제어, 일부 주소 변경
POSTROUTING	패킷의 최종 출구가 정해진 뒤, 실제 나가기 직전	출발지 주소 변경 (SNAT/MASQUERADE)

로컬 머신으로 향하는 트래픽은 input 경로에서 prerouting과 input 훅을 거치고, 이후 로컬 프로세스가 생성한 트래픽은 output과 postrouting 경로를 따릅니다. 라우터로 동작하도록 설정한 경우, 로컬로 향하지 않는 패킷들은 forward 훅에서 보이며, 이런 포워딩된 패킷은 prerouting → forward → postrouting 순서로 흐릅니다.



7.2 iptables 기본 개념

iptables는 이 Netfilter혹들 위에 테이블(table) → 체인(chain) → 규칙(rule) 구조를 엮은 사용자 공간 도구입니다.

iptables is the userspace command line program used to configure the Linux 2.4.x and later packet filtering ruleset. It is targeted towards system administrators. Since Network Address Translation is also configured from the packet filter ruleset, iptables is used for this, too. - netfilter.org — The netfilter.org "iptables" project

주요 테이블:

테이블	용도	주로 참조하는 계층	주로 쓰이는 체인
filter	패킷 허용/차단 (기본 방화벽)	L3/L4	INPUT, FORWARD, OUTPUT
nat	주소/포트 변환 (SNAT/DNAT)	L3(주소)+L4(포트)	PREROUTING, POSTROUTING, OUTPUT
mangle	패킷 헤더 수정(TOS, TTL, mark 등)	L3	5개 체인 모두
raw	conntrack 추적 여부 결정 (NOTRACK)	L3/L4	PREROUTING, OUTPUT
security	SELinux 등 MAC(강제접근제어) 연동	L3/L4	INPUT, FORWARD, OUTPUT

NAT의 종류:

종류	설명	관여 계층	훅 위치
SNAT (Source NAT)	출발지 주소 변경 (사설→공인)	L3	POSTROUTING
DNAT (Destination NAT)	목적지 주소 변경 (포트포워딩)	L3(+L4 포트)	PREROUTING
Masquerade	동적 SNAT (공인 IP가 유동적일 때)	L3	POSTROUTING
Redirect	로컬 프록시로 리다이렉트	L3/L4	PREROUTING/OUTPUT

7.3 Netfilter + iptables 예시

Example 1. 웹 서버의 HTTP(80) 접속 차단

- INPUT 체인 + filter 테이블

```
iptables -A INPUT -p tcp --dport 80 -j DROP`
```

- **INPUT** : 이 서버가 최종 목적지인 패킷
- **-p tcp --dport 80** : TCP 80번 포트(HTTP)
- **DROP** : 응답 없이 차단

Example 2. 내부 사설망 - 외부에서의 NAT

- 공유기 / 게이트웨이 서버가 **192.168.1.0/24** 내부 장비들의 트래픽을 인터넷으로 보내며, 출발지 IP를 자신의 공인 IP로 바꿔야 함
- **POSTROUTING** 체인 + **nat** 테이블

```
iptables -t nat -A POSTROUTING -s 192.168.1.0/24 -o eth0 -j MASQUERADE
```

- - **-t nat** : 주소 변환용 테이블
- **POSTROUTING** : 외부로 나가기 직전
- **-s 192.168.1.0/24** : 내부 사설 IP 대역에서 온 패킷
- **-o eth0** : 인터넷 쪽 네트워크 인터페이스
- **MASQUERADE** : 출발지 IP를 **eth0**의 현재 IP로 바꿈

Example 3. 외부 8080 포트를 내부 웹 서버 80 포트로 연결 (포트 포워딩)

- 공유기 또는 리버스 프록시 서버가 외부의 **8080** 요청을 내부 웹 서버 **192.168.1.100:80**으로 전달하는 포트 포워딩
- **PREROUTING** 체인 + **nat** 테이블

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 8080 \
-j DNAT --to-destination 192.168.1.100:80
```

- **PREROUTING** : 패킷이 들어온 직후, 목적지 기반 라우팅을 하기 전
- **DNAT** : 목적지 IP와 포트를 변경
- 외부 **공인IP:8080** 요청이 내부 **192.168.1.100:80**으로 바뀜

7.3 nftables

최신 배포판(RHEL 8+, Debian 10+ 등)은 기본 방화벽 도구를 **nftables**로 전환했습니다. nftables는 iptables와 동일한 Netfilter 인프라(혹, conntrack, NAT 엔진, 로깅)를 재사용하지만 패킷 분류 프레임워크가 새로 설계되었습니다.

In a major change from iptables, which predefines chains at every hook (i.e. INPUT chain in filter table), nftables predefines no chains at all. - [nftables wiki — Netfilter hooks](#)

nftables는 iptables처럼 테이블과 체인이 정해진 형태로 미리 존재하지 않습니다. nftables는 사용자가 테이블과 체인을 만들고 Netfilter의 어느 시점에 붙일지를 지정하는 방식입니다.

예를 들어, 서버로 들어오는 간단한 SSH와 웹 접속만 허용하는 간단한 nftables 규칙을 보겠습니다.

```
table inet firewall {
  chain input {
    type filter hook input priority 0;
    policy drop;

    ct state established,related accept
    iif lo accept
    tcp dport { 22, 80, 443 } accept
  }
}
```

- `table inet firewall`: IPv4와 IPv6에 공통 적용할 테이블
- `chain input`: 서버로 들어오는 패킷을 다루는 체인
- `hook input`: Netfilter의 `INPUT` 실행 시점에 연결
- `policy drop`: 아래 허용 규칙에 맞지 않으면 기본 차단
- `ct state established,related accept`: 기존 연결의 응답 트래픽 허용
- `iif lo accept`: 로컬 루프백 통신 허용
- `tcp dport { 22, 80, 443 } accept`: SSH, HTTP, HTTPS 허용

iptables와 차이점을 비교하면 다음과 같습니다.

항목	iptables	nftables
설계	IPv4, IPv6, ARP, 브리지용 명령 체계가 분리되어 발전	하나의 통합 프레임워크
IPv4/IPv6 공통 규칙	<code>iptables</code> , <code>ip6tables</code> 를 별도 관리	<code>inet</code> 테이블 하나로 공통 관리 가능
규칙 적용	명령을 한 줄씩 실행	ruleset을 한 번에 로드 가능

항목	iptables	nftables
변경 중 일관성	여러 명령 사이에 잠깐 정책이 불완전할 수 있음	전체 규칙 세트를 원자적으로 교체 가능
다수 IP·포트 처리	규칙 여러 개 또는 <code>ipset</code> 조합이 흔함	<code>set</code> , <code>map</code> , <code>verdict map</code> 을 기본 기능으로 제공
문법	옵션 중심이고 오래된 문법	패킷 조건 → 동작 구조로 비교적 일관됨
확장성	복잡한 정책에서 규칙 수가 늘기 쉬움	집합·맵으로 규칙을 압축하기 쉬움

정확히 기억 나지는 않지만 nftables는 iptables보다 조회 성능이 더 뛰어나다는 얘기를 들은 적이 있어서, 함께 알아보았습니다.

우선 iptables는 전통적으로 규칙을 위에서 아래로 **순서대로** 비교합니다. 따라서 규칙이 많으면 뒤쪽 규칙을 찾기까지 많은 비교가 필요할 수 있습니다.

반면에 nftables는 IP, 포트, 프로토콜 등을 `set` 이나 `map` 자료구조로 묶어 매우 효율적으로 조회할 수 있습니다.

예를 들어, 아래와 같이 허용 포트가 수백 개라고 가정해보겠습니다.

```
tcp dport { 22, 80, 443, ...100개... 8080 } accept
```

iptables는 규칙을 순서대로 확인하는 반면, nftables는 `set` 과 같은 자료구조로 빠르게 찾을 수 있기 때문에 효율적입니다.

8. L4 전송 계층 (Transport Layer)

L4는 포트 번호를 이용해 하나의 IP 위에서 여러 애플리케이션의 트래픽을 다중화하고, 필요하다면 신뢰성 있는 전달을 보장하는 계층입니다.

- **TCP**: 연결 지향, 순서 보장, 재전송을 통한 신뢰성 있는 전달. 3-way handshake로 연결을 수립합니다.
- **UDP**: 비연결형, 순서·신뢰성 보장 없이 빠르게 전달. DNS 조회, 스트리밍, QUIC 등에 사용됩니다.
- **conntrack**과의 관계: 앞서 7.5절에서 다룬 conntrack은 L4 포트 정보를 5-tuple의 일부로 사용해 연결 상태를 추적하므로, L4는 Netfilter의 상태 추적 기능과 떼어 수 없는 관계에 있습니다.

이 계층은 소켓 API(`socket()`, `bind()`, `listen()`, `connect()`)를 통해 유저 공간 애플리케이션에 노출되며, 커널은 각 소켓에 대해 송수신 버퍼, 혼잡 제어(congestion control) 알고리즘 등을 관리합니다.

9. L5~L7 세션/표현/응용 계층

L5(세션), L6(표현), L7(응용) 계층은 TCP/IP 4계층 모델에서는 하나의 "응용 계층"으로 통합되는 경우가 많으며, Linux 커널 네트워크 스택 자체보다는 유저 공간 라이브러리와 애플리케이션이 담당하는 영역입니다.

- **L5 세션 계층:** 소켓이 유지되는 동안의 논리적 세션 관리. TLS 세션 재사용(session resumption), HTTP Keep-Alive 등이 여기에 해당합니다.
- **L6 표현 계층:** 데이터의 인코딩, 압축, 암호화. TLS/SSL이 대표적이며, Linux에서는 OpenSSL이나 커널 TLS(kTLS) 오피로드 기능으로 지원됩니다.
- **L7 응용 계층:** HTTP, DNS, SSH 등 실제 애플리케이션 프로토콜. Linux 커널이 직접 개입하기 보다는 유저 공간 데몬(nginx, sshd, bind 등)이 처리하지만, 일부는 커널의 `AF_XDP` 나 `sendfile()` 같은 시스템 콜을 통해 가속화됩니다.

이 계층들은 이 문서에서 다룬 커널 네트워크 스택(L2~L4)의 결과물을 소비하는 최종 소비자라고 볼 수 있습니다.

부록: 컨테이너 네트워킹

Docker의 기본 네트워크 모델을 예로 들면, 지금까지 다룬 계층별 개념이 아래처럼 하나로 합쳐집니다.

1. 컨테이너 생성 시 새로운 **network namespace** 생성 (전 계층 격리)
2. **veth pair**(L2) 생성, 한쪽은 컨테이너 netns에, 다른 쪽은 호스트에 남김
3. 호스트 쪽 veth를 **docker0 bridge**(L2)에 연결
4. 컨테이너가 외부와 통신하려면 호스트가 **iptables MASQUERADE**(L3/L4)로 SNAT 수행
5. 외부에서 컨테이너 포트로 접근(`-p`) 시 **iptables DNAT**(L3/L4)로 포워딩
6. 호스트의 라우팅 테이블(L3)이 이 모든 패킷의 다음 hops를 결정
7. 컨테이너와 브리지 간 프레임 전달 시 **ARP**(L2/L3 경계)가 MAC 주소를 해석

Kubernetes CNI 플러그인들(bridge, Calico, Cilium, flannel 등)도 결국 이 조합을 각자의 방식(라우팅 기반 vs 오버레이 vs eBPF 기반)으로 확장한 것입니다.

참고 자료

- [ip-netns\(8\)](#) — 네트워크 네임스페이스 관리
- [veth\(4\)](#) — Virtual Ethernet Device 매뉴얼
- [Ethernet Bridging](#) — The Linux Kernel documentation
- [arp\(7\)](#) — Linux manual page
- [Netfilter's flowtable infrastructure](#) — The Linux Kernel documentation
- [Family conntrack netlink specification](#) — The Linux Kernel documentation
- [Networking](#) — The Linux Kernel documentation

RFC

- [RFC 826](#) — An Ethernet Address Resolution Protocol (ARP)

Netfilter / iptables / nftables 프로젝트

- [netfilter.org](#) — iptables 프로젝트 공식 페이지
- [netfilter.org](#) — 공식 문서 인덱스
- [nftables wiki](#) — Netfilter hooks
- [Netfilter](#) — Wikipedia (역사 및 개요)